

Alimentation d'un Data Warehouse scolaire

Apache Airflow, Apache Spark et MongoDB

Projet – Ingénierie de données

Septembre 2026

Résumé

Ce rapport décrit la conception et la réalisation d'un **entrepôt de données (Data Warehouse)** à partir d'une base scolaire de gestion PostgreSQL. Un **DAG Airflow** déclenché quotidiennement reconstruit le modèle dimensionnel (six dimensions, une table de faits, deux agrégats) dans **MongoDB**. Les transformations lourdes et les agrégations sont déléguées à un cluster **Apache Spark** exécuté en conteneurs. Un **contrôle de qualité** portant sur les valeurs manquantes protège le chargement par un mécanisme de court-circuit. L'ensemble est conteneurisé avec Docker Compose et reproduit les étapes ETL *extraction – transformation – chargement* de manière automatisée et auditable.

1 Contexte et objectifs

Un établissement gère ses données de scolarité (étudiants, classes, matières, professeurs, notes) dans une base PostgreSQL conçue pour l'*exploitation*. Ce système, bien adapté aux écritures du quotidien, l'est moins à l'*analyse* : schéma fortement normalisé, requêtes analytiques coûteuses, absence d'historisation et d'indicateurs prêts à l'emploi.

Le projet répond à trois objectifs :

1. construire un entrepôt de données (Data Warehouse) reflétant l'activité scolaire, selon un modèle *dimensionnel en étoile* ;
2. automatiser son alimentation par un pipeline ETL planifié et robuste ;
3. garantir la *qualité* des données chargées (valeur ajoutée du processus).

2 Architecture globale

L'architecture (figure 1) sépare la **source** (PostgreSQL), l'**orchestration** (Airflow), le **calcul distribué** (Spark) et la **cible** (MongoDB). Tous les services tournent en conteneurs Docker, à l'aide d'un `docker-compose.yaml` unique.

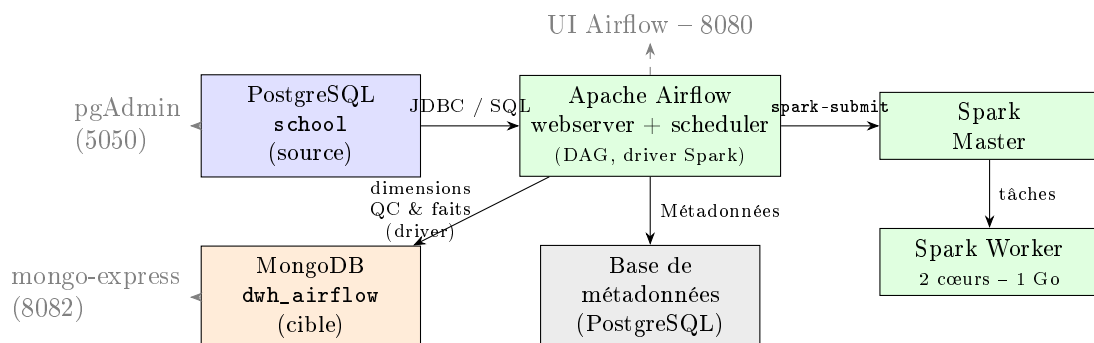


FIGURE 1 – Architecture du pipeline ETL (ports d'accès hôtes entre parenthèses).

Deux bases PostgreSQL coexistent : le *moteur source* `school` (données de gestion) et la *base interne* d'Airflow (métadonnées, DAGs, connexions). Le cluster Spark est en mode **standalone** : un master (port 7077) et un worker aux ressources volontairement modestes (2 cœurs, 1 Go). L'interface web Spark répond sur le port 8081.

Choix des ports. Les ports 5432 et 27017 étant utilisés par des installations locales, la source et la cible du projet sont exposées respectivement sur 5433 et 27018. L'isolation ainsi garantie assure qu'aucune instance locale n'est modifiée par le pipeline.

3 Modèle dimensionnel

Le schéma cible respecte le modèle *en étoile* : le fait `fait_notes` porte les mesures et référence six dimensions (tableau 1).

Objet	Collection	Contenu / cardinalités
Dimensions	6 collections <code>dim_*</code>	étudiants (1000), enseignants (60), matières (30), classes (24), jours (289), types (2).
Faits	<code>fait_notes</code> (23 940)	une ligne par note : <code>note</code> , <code>credit</code> + clés <code>id_Dim*_FK</code> .
Agrégats	<code>agg_moyenne_par_matiere</code> (30) <code>agg_moyenne_par_etudiant</code> (1000)	moyenne des notes par matière. moyenne par étudiant.
Qualité	<code>qc_report</code> (1)	rapport de contrôle du dernier run.

TABLE 1 – Collections produites par le pipeline.

Les identifiants de substitution (*surrogate keys*) sont générés à la charge : les références temporelles et de type d'évaluation sont reconstruites dans Spark à l'aide de regroupements *dense rank*. *Remarque importante* : la table source NOTE possède un `id_professeur` systématiquement vide ; la dimension enseignant n'est donc pas référencée par le fait (documenté, choix assumé).

4 Pipeline d'intégration (DAG)

Le DAG `dwh_scolaire_postgres_to_mongo` est planifié à `@daily` (`catchup=False`), chaque tâche supportant un nouvel essai après 5 minutes. L'enchaînement (figure 2) comprend neuf tâches.

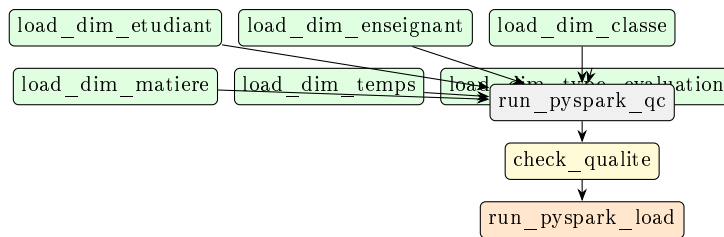


FIGURE 2 – Enchaînement des tâches du DAG.

Étape 1 – Dimensions (SQL). Six tâches *PythonOperator* lisent les tables de gestion via un hook PostgreSQL, transforment chaque ligne en document JSON et écrivent par `replace_many` (rechargement total) dans la collection cible. Les dimensions sont stables et peu volumineuses : un traitement SQL dédié suffit.

Étape 2 – Contrôle qualité (PySpark). Le job `spark.py` (mode `qc`) compte les notes, les valeurs manquantes et calcule un taux. Il consigne le résultat dans `qc_report`. Avec le jeu fourni : 23 940 notes, dont 730 valeurs absentes, soit un taux de 3,05 % pour un seuil de 20 % : statut **OK**.

Étape 3 – Garde-fou. `check_qualite`, un *ShortCircuitOperator*, relit le rapport : en l'absence de rapport ou en cas d'*anomalie* déclarée, il interrompt le DAG ; toutes les tâches aval sont marquées `skipped`. Aucune donnée douteuse ne peut atteindre le chargement des faits.

Étape 4 – Chargement et agrégats (PySpark). Le mode `load` relit NOTE par **JDBC partitionné** (colonne `id_matiere`, jusqu'à quatre partitions en parallèle), joint les référentiels (matière, étudiant, classe), reconstruit les clés de substitution et écrit `fait_notes` puis les deux collections d'indicateurs. L'écriture est réalisée *depuis le driver* avec le client `pymongo`, par lots de 1 000 documents, toujours en rechargement total (drop puis insert).

5 Résultats

Deux exécutions complètes du DAG ont été mesurées (déclenchement manuel et run planifié) :

- **dimensions** : 6 collections chargées (cardinalités du tableau 1) ;
 - **faits** : fait_notes (23 940 documents) ;
 - **indicateur par matière** : agg_moyenne_par_matiere (30) ;
 - **indicateur par étudiant** : agg_moyenne_par_etudiant (1000) ;
 - **qualité** : rapport OK (3,05 % de valeurs manquantes < seuil de 20 %) ; la barrière qualité a été vérifiée en abaissant provisoirement le seuil : le chargement des faits a bien été court-circuité ;
 - **durées observées** (poste local) : contrôle qualité $\approx$ 1 min 30, chargement des faits $\approx$ 4 min.
- Le pipeline est **reproductible** (détail en section 6). Deux interfaces de supervision complètent la livraison : pgAdmin (5050) et mongo-express (8082).

6 Mise en œuvre et reproductibilité

L’environnement se déploie en trois commandes. L’initialisation de la base source est automatique au premier démarrage (rejeu du script data/G2.sql).

```
# 1. Construire l’image Airflow et démarrer toute la pile
docker compose up -d

# 2. Initialiser Airflow (base centrale + utilisateur admin)
docker compose up airflow-init --wait

# 3. Recréer les connexions (source, cible, Spark)
docker compose exec airflow-scheduler bash /opt/airflow/scripts/init_connections.sh
```

Le DAG est ensuite activé puis déclenché depuis l’interface Airflow (port 8080). Les échanges avec la source et le cluster reposent sur trois connexions Airflow (tableau 2).

Connexion	Type	Cible
postgres_default	PostgreSQL	postgres://school:school@postgres-source:5432/school
mongo_default	MongoDB	mongodb://mongodb-dwh:27017
spark_default	Spark	host spark://spark-master, port 7077

TABLE 2 – Connexions Airflow utilisées par le pipeline.

7 Limites et perspectives

Les limites, documentées dans le rapport technique du dépôt, sont assumées :

- **Rechargement total** : simple et cohérent, mais non incrémental ; un horodatage de modification permettrait par la suite une charge incrémentale.
- **Écriture côté driver** : les exécuteurs ne disposent pas de pymongo ; le connecteur Spark-MongoDB officiel déplacerait cette écriture vers les exécuteurs.
- **Identifiants de connexion** visibles dans la ligne de commande `spark-submit` (acceptable en local ; un gestionnaire de secrets est prévu en production).
- **Agrégat global** (`dense_rank`) ramène temporairement la donnée sur une partition unique ; à grande échelle, une partition par clé de substitution s’imposerait.
- **Volume traité** : modeste par choix ; l’architecture (lecture partitionnée, scale-out Spark) est conçue pour monter en charge sans changement de code.

8 Conclusion

Le projet démontre la mise en œuvre complète d’une chaîne ETL moderne : orchestration planifiée (Airflow), calcul distribué (Spark/PySpark), transformation vers un modèle dimensionnel et chargement dans un entrepôt document MongoDB. Le contrôle qualité intégré et le court-circuit associé apportent la garantie métier que seules des données conformes alimentent l’entrepôt. La conteneurisation rend l’environnement reproductible et portable. Les pistes d’évolution identifiées (chargement incrémental, connecteur Spark officiel, secret management) dressent un plan d’amélioration réaliste vers une exploitation de production.